E-RAKSHAK HACKATHON 2026 · PROBLEM STATEMENT PS-08

Surat Traffic Intelligence

Data-Driven Traffic Optimization & Adaptive Infrastructure Intelligence

Real-time lane density from CCTV · adaptive signal optimisation · bottleneck detection
— with two signal controllers, each validated on data that can actually support it.

CAMERAS	ZONES	WINDOWS	CHECKS
7	21	2,451	18/18

TEAM
[TEAM NAME]
[Member 1] · [Member 2] · [Member 3] · [Member 4]

PROBLEM STATEMENT

PS-08 — what is being asked

Use the city's existing CCTV network to measure, optimise and predict — without new hardware.

- **Measure traffic density** per lane, from existing CCTV feeds
- **Optimise signal timing** adaptively, from what is actually measured
- **Detect bottlenecks** and identify where they form
- **Recommend improvements** to infrastructure, with evidence

WHAT WE WERE GIVEN

Source	7 Surat CCTV clips
Resolution	1920×1080 @ 20 fps
Duration	≈31.5 min total
Ground truth	none
Camera calibration	none
Signal timing data	none

The constraint that shaped everything

No surveyed reference ships with this data, so no pixel-to-metre scale can be recovered from it. The system separates what it measures from what it estimates, rather than quietly presenting one as the other.

PROBLEM UNDERSTANDING

What this data can — and cannot — support

Reading the footage before writing the system changed the design twice.

Camera	Scene	Mode	What it can support
GauravPath	Divided arterial, marked lanes	junction	Lane density, signal timing
Rokadiya Hanuman	Divided multi-lane road	junction	Lane density, signal timing
Chauta Bazaar	Dense market street	area	Occupancy, crowding, alerts
Mahidharpura	Near-total gridlock	area	Occupancy, crowding, alerts
Gopi Talav	Pedestrian-dominant gate	area	Occupancy, crowding, alerts
Rly Station	Station forecourt	area	Occupancy, crowding, alerts
Central Bus Depo	Depot platform	area	Occupancy, crowding, alerts

Only 2 of 7 cameras can support the requested measurement

Five cameras watch market streets, a station forecourt and a bus depot — no lane discipline, no signal to optimise. "Lane density" is not a measurable quantity there, so forcing it would mean inventing numbers. Those run in area mode instead.

In the worst gridlock, detection counts fewer vehicles

The failure mode that would have inverted every ranking in the system — silently.

Camera	Detections	Actual condition
Rokadiya Hanuman	7,664	flowing arterial
Central Bus Depo	2,905	busy
GauravPath	1,957	free-flowing
Mahidharpura	463	worst gridlock in dataset

- **Cause.** At extreme density a wall of overlapping vehicles stops being separable and the detector's recall collapses.
- **Risk.** A naive "more detections = more congestion" system would rank Surat's worst street among its emptiest — confidently, and backwards. Every number would look plausible.
- **Fix.** A texture-based occupancy index (Canny edge density against absolute cross-camera bounds) which has no notion of an object, so occlusion cannot defeat it.

Result

Mahidharpura now flags 452 of 453 windows as saturated and ranks correctly as the most congested site. The occupancy index is normalised against absolute bounds measured across all seven cameras, so a value of 0.8 means the same thing on every camera.

PROPOSED SOLUTION

Two controllers, two datasets, one hard boundary

Surat's junctions run Webster's. Max-pressure is validated on FLUID. Neither borrows from the other.

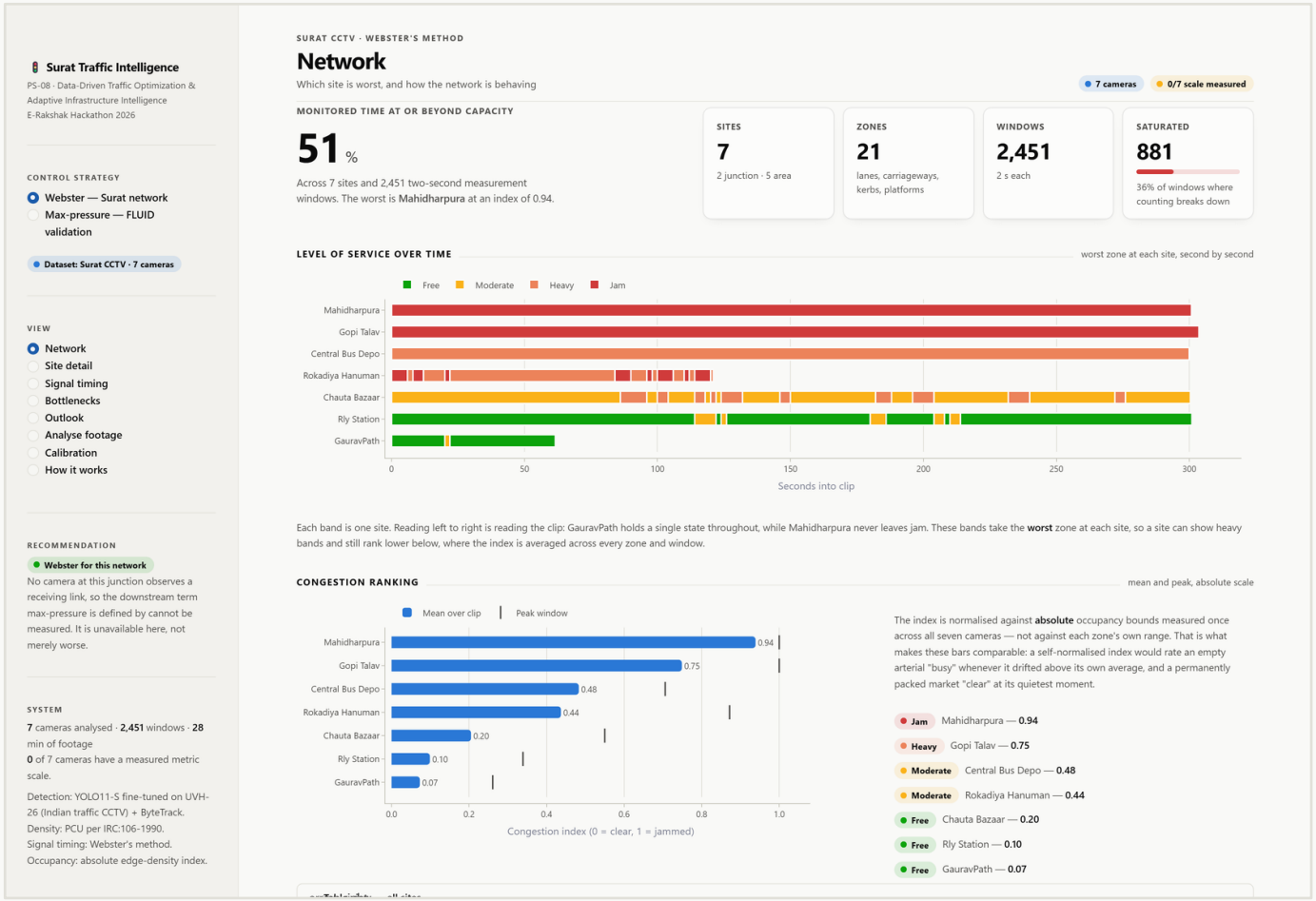
	Webster's	Max-pressure (Varaiya 2013)
Needs to observe	arrival flow per approach	approach queue AND the queue it discharges into
Cameras required	one per approach	one per approach + one per receiving link
Surat CCTV provides	✓ exactly this	✗ no camera watches a receiving link
Runs on	the 7 Surat clips — every operational figure	FLUID: 14 drone recordings, 3 real intersections
On Surat it reports	cycle, splits, delay, saturation	greedy-queue — and refuses to name a phase

Why not just drop the missing term?

Because that does not give a degraded max-pressure — it gives greedy queue control, which Varaiya's §3.1 Example 2 proves fails to stabilise demands max-pressure handles. So rather than fake it, a dataset containing the missing term was obtained, and the controller is validated there.

KEY FEATURES

One dashboard, two tracks, thirteen views



TRACK 1 — WEBSTER'S · SURAT CCTV

- Network — LOS bands and congestion ranking
- Site detail — zones, overlay, heatmap, trends
- Signal timing — cycle, splits, delay, saturation
- Bottlenecks — every alert with its evidence
- Outlook — forecasts with prediction bands
- Analyse footage · Calibration · How it works

TRACK 2 — MAX-PRESSURE · FLUID DRONE

- Validation — replayed against 3 real signals
- Scene — aerial with live queue shading
- Phase pressure — every movement's terms
- Head-to-head — 3 demand regimes simulated
- Method — what the data can and cannot say

TECHNOLOGY STACK

Chosen for what the data needs

Layer	Choice	Why this one
Detector	YOLO11-S	Better on small, distant, partly occluded vehicles
Weights	UVH-26 fine-tune	Indian CCTV domain; 14 classes incl. auto-rickshaw
Tracker	ByteTrack	Associates low-confidence detections in crowds
Density unit	PCU · IRC:106-1990	The unit Indian capacity standards are defined in
Signal method	Webster's + HCM delay	Needs only what one camera per approach sees
2nd controller	Max-pressure (Varaiya)	Throughput-optimal — gated on observability
Saturation flow	IRC:SP:41-1994, S = 525·W	Width-based; lane count is unrecoverable here
Occupancy	Canny edge density	No object resolution needed — survives saturation
Dashboard	Streamlit + Plotly	Operator view without a JS build step
Language	Python 3.11 · PyTorch 2.5 · OpenCV	GPU-accelerated, standard

Hardware: NVIDIA RTX 3050 6 GB laptop GPU. No cloud training was needed — the UVH-26 weights removed the training step entirely.

SYSTEM ARCHITECTURE DIAGRAM

From video frame to signal recommendation

1 DETECT + TRACK
detector.py
YOLO11-S · ByteTrack

2 LOCATE
zones.py
lane / zone polygons

3 MEASURE
metrics.py
two parallel paths

DETECTION PATH
PCU density, speed, queue, flow (q = k·v)

OCCUPANCY PATH
texture-based index — survives saturation

4 OPTIMISE
signal_opt.py · maxpressure.py

5 BOTTLENECKS
bottleneck_detect.py — 5 patterns

6 PREDICT
forecast.py — damped trend + band

7 PRESENT dashboard.py — two tracks, 13 views

Standards, not guesses.
IRC:106-1990 (PCU) · IRC:SP:41-1994 (saturation flow) · IRC:103-2012 (pedestrian green)

WORKFLOW / PROCESS FLOW

End to end, in five steps

One command runs the dashboard; the full pipeline is five steps from raw video.

01 **Reference frames**
One clean frame per camera, for every overlay drawn later.

02 **Calibration**
Lane / zone polygons traced, then validated against a movement heatmap.

03 **Analyse**
Detect → track → assign to zone → measure. Writes one row per zone per 2 s window.

04 **Optimise & detect**
Webster's cycle and splits; five sustained-pattern bottleneck detectors; damped-trend forecast.

05 **Present**
Two-track dashboard. Every chart has a table beside it, so no value is reachable only by hovering.

Reproducible without the video

The metrics CSVs are committed, so the entire Webster track — every operational figure in this deck — runs immediately with no video download, no GPU and no model weights.

AI / ML MODELS

The weights matter more than the architecture

Model	UVH-26-MV-YOLOv11-S
Source	HuggingFace iisc-aim/UVH-26 @ v1.0
Trained on	26,646 annotated 1080p frames
Domain	Indian Safe-City CCTV
Classes	14 India-specific vehicle types
Tracker	ByteTrack (crowded-scene association)
Throughput	≈12 FPS @ 960px on RTX 3050 6 GB

- **Why this fine-tune.** Same domain as our footage, and it carries the auto-rickshaw — a class COCO does not have at all, and which dominates most of these clips.
- **We did not train it.** Used as published. That removed the need to train on an unlabelled dataset.
- **Density in PCU, not counts.** Twelve two-wheelers and twelve buses are not the same amount of traffic. IRC:106-1990 factors.

Flow is never counted from tracker IDs

ByteTrack reissues IDs whenever a track drops briefly — constant in dense mixed traffic — so counting them inflated flow tenfold, to 12,000–23,000 PCU/hr against a real saturation flow near 1,800. Flow comes from the fundamental relation q = k·v instead, on directly measured density and speed.

The operational system — Webster's

Baseline is an untuned fixed timer with equal green splits.

WORST-APPROACH SATURATION

1.01 → 0.929

Crosses back under 1.0 — the queue clears within the cycle again

CYCLE LENGTH

135.6 s

Optimal, against a 120 s fixed baseline

MEAN DELAY

−3.8 %

53.5 → 51.4 s/PCU, HCM control delay

EVIDENCE PRODUCED

31 alerts

plus 21 zone forecasts with prediction intervals

#	Site	Mode	Congestion	Saturated
1	Mahidharpura – Diamond Mkt	area	0.939	452 / 453
2	Gopi Talav Gate	area	0.749	167 / 456
3	Central Bus Depo	area	0.483	213 / 450
4	Rokadiya Hanuman	junction	0.437	0 / 244
5	Chauta Bazaar	area	0.205	4 / 453

TWO INDEPENDENT CHECKS

- **Cycle sweep.** Empirical delay minimum lands at 130 s against Webster's closed form of 135.6 s.
- **Equal saturation.** The optimal plan equalises degree of saturation across approaches (0.929 / 0.929) — the known optimality condition.

RESULTS · FLUID

Max-pressure, validated on real signals

Replaying max-pressure against what each real fixed-time signal actually did.

Intersection	Cycle	Real signal: green with nothing to relieve	Max-pressure
FI (4-way, 3-phase)	96 s	41.1%	1.5%
FIDRT (dedicated right turns)	123 s	66.3%	2.3%
TI (T-intersection)	71 s	46.6%	16.0%

SIMULATED COUNTERFACTUAL

Scenario	Fixed	Max-pr.
Steady, balanced	18.6	20.8
Demand swings	24.5	18.5
Downstream blockage	150.0	102.9

The operator's decision rule — narrower than the literature's enthusiasm suggests

Max-pressure earns its place when the split of demand moves between approaches, and above all when something downstream backs up (−31% delay, spillback 289 s → 88 s). On steady demand it is the worse choice — it pays an inter-green for a benefit that is not there. A system offering it as a straight upgrade would be selling the wrong thing.

Deliberately not a delay-reduction claim. Had max-pressure actually run, the queues would have evolved differently — that is an observation about the recorded signals, made from the recorded queues. The counterfactual needs the simulator above.

INNOVATION AND UNIQUE VALUE PROPOSITION

Four things this system does that a standard pipeline does not

The observability gate

Max-pressure refuses to name a phase when its downstream term is unmeasurable, and returns the weaker answer under its own name — greedy-queue — so a caller cannot present it as max-pressure. The refusal is structural, not a warning label.

Occlusion-proof measurement

A texture-based occupancy index with no notion of an object, so the gridlock that collapses detection cannot invert the ranking. This is the difference between a plausible answer and a correct one.

Separated measurement from estimation

Every figure carrying the uncalibrated pixel scale is marked and banner-flagged; the ranking, level of service and the entire signal comparison are scale-free and unaffected. Every headline result is in the scale-free category.

Documentation that cannot drift

The verification suite recomputes every figure quoted in the README and report, and fails if a number changes in the code but not the docs — or the reverse.

CHALLENGES FACED

Eleven bugs found and fixed — four worth showing

Each was caught by a check that did not trust the output.

#	Symptom	Root cause	How it was caught
1	Speeds 5× too low (4–8 km/h on a free arterial)	Isotropic pixel-to-metre scale — vertical and horizontal pixels treated alike	Physical implausibility of the reported speed
2	Flow 10× too high (12,000–23,000 PCU/hr)	Counting new ByteTrack IDs as arrivals; IDs churn constantly in dense traffic	Compared against real saturation flow ≈1,800
3	"Optimised" plan was worse (−9.5%, delay increased)	Flow ratio computed per critical lane, saturation per whole approach — mixed units	Swept cycle length; the delay curve had no minimum, which is impossible
4	Max-pressure named a phase it had no data for	Missing downstream queue silently treated as zero	Audit against Varaiya §3.1 — the result was a different controller

The pattern

Three of these four produced numbers that looked entirely reasonable on screen. What caught them was checking outputs against physical limits and known theory — not testing that the code ran.

FUTURE SCOPE

What one measurement, and what more data, would unlock

IMMEDIATE — NO NEW HARDWARE

- **One surveyed road width.** Promotes every metric figure from estimate to primary. The Calibration view does it in ~2 minutes without a site visit.
- **Cameras on receiving links.** Unlocks true max-pressure on Surat — the controller is already implemented and validated.
- **Continuous RTSP ingest.** Replaces 31 minutes of clips with a live feed; the pipeline is already windowed for it.

LONGER TERM

- **Seasonal forecasting.** Current model has minutes of history. Weeks of continuous operation would justify a daily/weekly seasonal model.
- **Coordinated corridors.** Green-wave offsets across adjacent junctions — the simulator already models two in series.
- **Production stack.** React + Kafka + PostGIS for city-scale deployment; the measurement core is unchanged.

STATED LIMITATIONS

- The metric scale is not calibrated — anything in km/h or PCU/km/lane is an estimate, and is labelled as one.
- Lane polygons are traced and heatmap-validated, not surveyed.
- The two junction cameras face different corridors, so the combined plan demonstrates the method on real measured demand rather than being deployable at one junction.
- FLUID is Chinese traffic — it validates the controller implementation; no claim from it transfers to Surat's fleet.